

DevOps Manual Module-3 | Course Code: BCSL657D | Credits: 01 | Semester: VI
CIE: 50 Marks | SEE: 50 Marks | Exam Hours: 100

MANGALURU — KARNATAKA

LABORATORY MANUAL

Ansible Configuration Management (Programs 7–8) Module-3

10.5281/zenodo.19641056

Project
Student Grade Tracker

Labs
10 Hands-On Sessions

Duration
~20 Hours Total

Programmes Covered

Programs 7–8
Jenkins CI/CD & Ansible
Config Mgmt

VTU Scheme
2021-22 onwards

Prepared By

Dr. Lokesh M R

Ph.D. (CSE) · M.Tech. (IT) · B.E. (ECE)
Professor,
Mangaluru

2026

Version 1.0
VTU Affiliated
Even Semester

Preface

This Laboratory Manual for DevOps (BCSL657D) has been prepared as a comprehensive, step-by-step practical guide for sixth-semester students of the B.E. programme affiliated to Visvesvaraya Technological University (VTU), Belagavi, under the 2021 scheme. The manual covers the full spectrum of DevOps tools and practices prescribed in the syllabus, spanning twelve structured experiments across build automation, continuous integration, configuration management, and cloud-based DevOps services.

The manual is organised into three broad thematic modules: (i) Build Automation using Apache Maven and Gradle (Programs 1–4), (ii) Jenkins CI/CD Pipeline (iii) Ansible Configuration Management (Programs 5–8), and (iv) Azure DevOps Services including Build and Release Pipelines (Programs 9–12). Each experiment begins with clear objectives and theoretical background, followed by step-by-step procedural instructions, expected outputs, and viva-voce questions to reinforce learning. *This manual contain ONLY (iii) Ansible Configuration Management (Programs 5–8), due to detail understanding purpose .*

This manual is aligned with the course outcomes (COs) defined in the VTU syllabus, enabling students to demonstrate hands-on proficiency in version control, continuous integration and deployment, configuration management, and cloud-based DevOps toolchains. The instructions are written keeping in mind students with varying levels of prior exposure to software engineering tools, and every effort has been made to ensure clarity and reproducibility.

I sincerely hope this manual serves as a valuable resource for students and faculty alike. Constructive feedback and suggestions for improvement are always welcome.

Dr. Lokesh M R
Professor

Purpose of This Manual

This laboratory manual has been designed as a comprehensive, hands-on guide for students and professionals who are beginning their journey into the world of DevOps. Unlike conventional theoretical textbooks, every concept presented here is grounded in practical application — using a single real-world project.

The DevOps movement represents a fundamental shift in how software is built, tested, deployed, and maintained. By bridging the gap between development and operations teams, DevOps practices enable organisations to deliver software faster, more reliably, and with greater quality. This manual introduces those practices in a structured, progressive manner — starting from environment setup on a Windows machine using WSL 2, and culminating in a fully automated CI/CD pipeline with container orchestration and production monitoring.

Who This Manual Is For

This manual is intended for undergraduate and postgraduate students of Computer Science and Engineering, Information Science, and related disciplines who are enrolled in a DevOps or Cloud Computing laboratory course. It is equally valuable for working professionals who wish to upskill in modern software delivery practices. A basic familiarity with the Linux command line and Python programming is assumed, but no prior DevOps experience is required.

How to Use This Manual

Each of the five labs in this manual builds directly upon the previous one. It is strongly recommended that students complete the labs in sequential order, as concepts and configurations introduced in earlier labs are prerequisites for later ones. Each lab is structured with clear objectives, step-by-step instructions with verified commands, explanatory notes, and a checkpoint section to confirm successful completion before moving forward.

All commands in this manual have been tested on Ubuntu 22.04 LTS running inside WSL 2 on Windows 10 (Build 19041 and later) and Windows 11. Code blocks with a dark background indicate commands to be executed in the Ubuntu/WSL 2 terminal. Where PowerShell or Windows commands are required, this is clearly stated.

Acknowledgements

This manual was developed as part of the DevOps Laboratory course [BCSL657D] at A J Institute of Engineering & Technology, Mangaluru, affiliated with Visvesvaraya Technological University (VTU), Belagavi. The author gratefully acknowledges the contributions of students, colleagues, and the broader open-source DevOps community whose feedback and resources made this work possible. Special thanks to the developers of WSL 2, Docker, GitHub Actions, Terraform, Ansible, Kubernetes, and the many open-source projects referenced throughout this manual.

A Note on Software Versions

Software versions change rapidly in the DevOps ecosystem. This manual was written and tested in early 2026 using the software versions specified at the beginning of each lab. While the core concepts remain consistent across versions, minor syntax differences may exist in newer releases. Students are encouraged to refer to official documentation when discrepancies arise. The Troubleshooting Guide at the end of this manual provides solutions to the most commonly encountered errors.

— Dr. Lokesh M R
Professor, Mangaluru

April 2026

Course Information

Detail	Information
Course Title	DevOps Laboratory
Course Code	BCSL657D
Scheme	2021 (VTU)
Semester	VI (Sixth)
Teaching Hours/Week	0:0:2:0 (L:T:P:S)
Credits	01
CIE Marks	50 (Journal: 30 Test: 20)
SEE Marks	50 (Scaled from 100)
Total Marks	100
Exam Hours	Minimum 02 Hours
Examination Type	Practical

Course Outcomes (CO)

CO	Description
CO2	Set up and configure Jenkins CI pipelines for automated build and artifact management.
CO3	Experiment with configuration management using Ansible.

Index of Experiments

Sl.	Experiment Title	CO Mapped
7	Configuration Management with Ansible: Basics of Ansible — Inventory, Playbooks, and Modules; Automating Server Configurations with Playbooks	CO3
8	Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins	CO2, CO3

Preface

Course Information

Course Outcomes (CO)

Index of Experiments

Program 7: Configuration Management with Ansible

7.1 Overview of Ansible

7.2 Prerequisites

Step 1: Update System and Install OpenSSH Server

7.3 Step 1: Create an Ansible User

7.4 Step 2: Configure SSH Key Authentication

7.5 Step 3: Install Ansible

7.6 Step 4: Create an Inventory File

7.7 Step 5: Test Connectivity with Ping Module

7.8 Step 6: Write a Sample Playbook

7.9 Step 7: Run the Playbook

7.10 Step 8: Verify the Result

7.11 Summary

Program 8: Jenkins CI Pipeline with Ansible Deployment

8.1 Overview

8.2 Step 1: Create a Simple Maven Project

8.3 Step 2: Create a Jenkins Freestyle Job

8.4 Step 3: Install Ansible Plugin in Jenkins

8.5 Step 4: Create Ansible Inventory File

8.6 Step 5: Write an Ansible Deployment Playbook

8.7 Step 6: Build the JAR and Prepare jenkins-artifacts

8.8 Step 7: Run the Jenkins Build

8.9 Step 8: Verify Deployment with Ansible

8.10 Common Errors and Fixes

Error 1: deploy.yml could not be found

Error 2: Could not find ~/jenkins-artifacts/myapp-1.0-SNAPSHOT.jar

Error 3: Permission denied on /home/ansible

Error 4: No POM in this directory

8.11 Summary

Program 8.0: Configuration Management with Ansible (Enhanced)

8.0.1 Removing Ansible and Cleaning Up

Step 1: Remove NGINX via Ansible Playbook

Step 2: Remove Ansible from the System

Step 3: Verify Ansible is Fully Removed

8.0.2 Summary

8.0.3 Viva-Voce Questions

Appendix Quick Reference

A. Common Jenkins Commands and URLs

B. Common Ansible Commands

C. Ansible State Values for Packages and Services

Program 7: Configuration Management with Ansible

Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook

Topics Covered: Basics of Ansible — Inventory, Playbooks & Modules • Automating Server Configurations • Writing and Running a Basic Playbook

7.1 Overview of Ansible

Ansible is an open-source IT automation tool used for configuration management, application deployment, and task automation. It uses a simple YAML syntax and communicates with managed nodes over SSH (agentless architecture), meaning no software needs to be installed on the target machines.

Concept	Description
Control Node	The machine where Ansible is installed and commands are run from.
Managed Node	The remote server(s) that Ansible manages.
Inventory	A file listing the managed nodes (hosts) and their groupings.
Playbook	A YAML file containing a set of tasks to execute on managed nodes.
Module	A reusable unit of work (e.g., apt, service, copy) used inside tasks.
Task	A single unit of work calling a module (e.g., install nginx, start a service).

7.2 Prerequisites

- Ubuntu Linux (or any Debian-based system) as the control node.
- SSH access to managed nodes.
- sudo privileges on the control node.

Step 1: Update System and Install OpenSSH Server

Terminal — Update & Install SSH

```
sudo apt update
sudo apt install openssh-server -y
```

Terminal — Start SSH Service

```
sudo systemctl start ssh
sudo systemctl enable ssh
sudo systemctl status ssh
# Press Ctrl+X to exit status view
```

7.3 Step 1: Create an Ansible User

It is recommended to create a dedicated user account for Ansible operations:

Terminal — Create and Configure the ansible User

```
# Navigate to home directory
cd ..

# Add the ansible user
sudo adduser ansible

# Add ansible user to sudo group
sudo usermod -aG sudo ansible

# Switch to ansible user
sudo su ansible
```

You will be prompted to set a password and fill in optional user information. Confirm all entries by typing Y when asked. After switching user, verify by checking the hostname:

Verify User Switch

```
hostname -I
```

7.4 Step 2: Configure SSH Key Authentication

Ansible uses SSH to communicate with managed nodes. Generate an SSH key pair and copy the public key to the target machine:

Generate SSH Key Pair

```
ssh-keygen
# Press Enter to accept defaults (no passphrase)
```

Copy the public key to the managed node (replace 10.0.2.15 with your target machine IP):

Copy SSH Key to Managed Node

```
ssh-copy-id ansible@10.0.2.15
# Type yes to accept the fingerprint
# Enter the ansible user password when prompted
```

After successful key copy, you should see: Number of key(s) added: 1

7.5 Step 3: Install Ansible

Install Ansible via APT

```
sudo apt update
sudo apt install ansible -y

# Verify installation
ansible --version
```

The output will display the Ansible version (e.g., ansible [core 2.16.3]), Python version, jinja2 version, and configuration paths.

7.6 Step 4: Create an Inventory File

The inventory file tells Ansible which hosts to manage. Create the Ansible configuration directory and inventory file:

Create Inventory

```
sudo mkdir -p /etc/ansible
sudo nano /etc/ansible/hosts
```

Add the following content to the file (for local testing using localhost):

/etc/ansible/hosts

```
localhost ansible_connection=local
```

Save and exit the nano editor: Ctrl+O, Enter to save, then Ctrl+X to exit.

Verify the inventory is parsed correctly:

Verify Inventory

```
ansible-inventory --list -y
```

7.7 Step 5: Test Connectivity with Ping Module

Use the Ansible ping module to verify that Ansible can reach all hosts in the inventory:

Ping All Hosts

```
sudo ansible all -m ping
```

Expected output for a successful connection:

Expected Output

```
localhost | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

NOTE

The expected output shows 'localhost' (not 'localhist'). Verify your /etc/ansible/hosts file contains the correct hostname entry.

7.8 Step 6: Write a Sample Playbook

A playbook is a YAML file that defines a series of tasks to automate. Create a playbook to install and start the NGINX web server:

Create the playbook file

```
nano install_nginx.yml
```

Paste the following YAML content:

install_nginx.yml

```
- name: Install and start NGINX on localhost
  hosts: local
  become: yes
  tasks:
    - name: Install NGINX
      apt:
        name: nginx
        state: present
        update_cache: yes
    - name: Ensure NGINX is running
      service:
        name: nginx
        state: started
        enabled: yes
```

Save and exit: CTRL+O, Enter, then CTRL+X.

IMPORTANT — Inventory Group Name Must Match Playbook

The hosts: field in your playbook MUST exactly match the group name in your inventory file.
Create ~/hosts with the following content:

```
[local]
localhost ansible_connection=local
```

If cat -A ~/hosts shows ^M at end of lines (Windows line endings), run:
sudo sed -i 's/\r//' ~/hosts

7.9 Step 7: Run the Playbook

Execute Playbook

```
ansible-playbook -i ~/hosts install_nginx.yml

# Verify inventory connectivity first:
ansible -i ~/hosts local -m ping
# Should return: localhost | SUCCESS => {"ping": "pong"}
```

You should see output showing each task being executed, ending with a PLAY RECAP showing ok=3, changed=0 or changed=2, failed=0.

7.10 Step 8: Verify the Result

Verify that NGINX was successfully installed and is responding:

Verify NGINX

```
curl http://localhost
```

If successful, you will see HTML output beginning with `<!DOCTYPE html>` and the NGINX welcome page content.

7.11 Summary

Program 7 — Key Takeaways

- Ansible uses an agentless SSH-based architecture for configuration management.
- The control node runs Ansible; managed nodes only need SSH and Python.
- Inventory files define the hosts Ansible manages (groups and individual hosts).
- Playbooks are YAML files containing named plays with tasks using modules.
- The ping module tests connectivity; apt and service modules manage packages and services.
- Run playbooks with: `ansible-playbook -i <inventory_file> <playbook.yml>`
- The group name in [brackets] in the inventory must exactly match hosts: in the playbook.

Program 8: Jenkins CI Pipeline with Ansible Deployment

Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins

Topics Covered: Set Up a Jenkins CI Pipeline for a Maven Project • Use Ansible to Deploy Artifacts Generated by Jenkins

8.1 Overview

This practical exercise integrates Jenkins and Ansible into a complete CI/CD workflow. Jenkins is used to build a Maven Java project and archive the resulting JAR artifact. Ansible is then used to deploy the artifact to the target server. This demonstrates a real-world DevOps pipeline combining CI (Jenkins) and Configuration Management/Deployment (Ansible).

Step	Action
Step 1	Create a simple Java Maven project with pom.xml.
Step 2	Configure a Jenkins Freestyle job to build the project.
Step 3	Archive the .jar artifact in Jenkins.
Step 4	Install the Ansible plugin in Jenkins.
Step 5	Configure Ansible Inventory and Playbook for deployment.
Step 6	Copy the built JAR to ~/jenkins-artifacts/.
Step 7	Trigger the Jenkins build and verify successful deployment.

8.2 Step 1: Create a Simple Maven Project

First, ensure Maven is installed on the system:

Install Maven (if not installed)

```
sudo apt update
sudo apt install maven -y

# Verify installation
mvn -version

# Also verify Java version
java -version
```

Create the project directory and generate a Maven project:

Create Maven Project

```
# Create and enter the project parent directory
mkdir ~/java-maven-app
cd ~/java-maven-app

# Generate a Maven project (creates the 'myapp' subfolder)
```

```
mvn archetype:generate \  
  -DgroupId=com.example \  
  -DartifactId=myapp \  
  -DarchetypeArtifactId=maven-archetype-quickstart \  
  -DinteractiveMode=false
```

```
# Navigate into the generated project  
cd myapp
```

IMPORTANT — pom.xml Must Exist Before mvn clean package

Running 'mvn clean package' from ~/java-maven-app will FAIL because pom.xml is inside the 'myapp' subfolder. Always cd into ~/java-maven-app/myapp first.

Error you will see if wrong directory:

[ERROR] The goal requires a project to execute but there is no POM in this directory.

Update pom.xml to use Java 11 (or match your installed Java version):

Update pom.xml — Set Java Version

```
nano ~/java-maven-app/myapp/pom.xml  
  
# Add/replace the properties section:  
# <properties>  
#   <maven.compiler.source>11</maven.compiler.source>  
#   <maven.compiler.target>11</maven.compiler.target>  
# </properties>
```

Full minimal pom.xml should look like:

pom.xml (minimal correct version)

```
<project>  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.example</groupId>  
  <artifactId>myapp</artifactId>  
  <version>1.0-SNAPSHOT</version>  
  <properties>  
    <maven.compiler.source>11</maven.compiler.source>  
    <maven.compiler.target>11</maven.compiler.target>  
  </properties>  
</project>
```

Build and verify the project locally:

Build and Verify Locally

```
cd ~/java-maven-app/myapp  
mvn clean package
```

```
# Confirm JAR was created:
```

```
ls target/  
# Expected: myapp-1.0-SNAPSHOT.jar
```

8.3 Step 2: Create a Jenkins Freestyle Job

1. From the Jenkins Dashboard (<http://localhost:8080>), click New Item.
2. Enter the project name: Build-Maven-Project
3. Select Freestyle project and click OK.
4. In the job configuration, under Build Environment, check Delete workspace before build starts.
5. Under Build Steps, click Add build step and select Execute shell.

Enter the following shell commands in the Execute shell box:

Build Step – Execute Shell

```
# Copy the Maven project from ansible home to Jenkins workspace  
cp -r /home/ansible/java-maven-app/myapp .
```

```
cd myapp
```

```
mvn clean package
```

Fix: Jenkins Permission Denied on /home/ansible/java-maven-app

Jenkins runs as the 'jenkins' user and cannot access /home/ansible/ by default.

Run these commands to grant Jenkins access:

```
sudo chmod o+rx /home/ansible  
sudo chmod -R o+rx /home/ansible/java-maven-app
```

OR add jenkins user to ansible group (recommended):

```
sudo usermod -aG ansible jenkins  
sudo systemctl restart jenkins
```

Then click Build Now again.

6. Under Post-build Actions, click Add post-build action and select Archive the artifacts.
7. In the Files to archive field, enter: myapp/target/*.jar
8. Click Save.

8.4 Step 3: Install Ansible Plugin in Jenkins

To trigger Ansible playbooks from Jenkins, install the Ansible plugin:

9. Go to Manage Jenkins > Plugins.
10. Click Available plugins.
11. Search for Ansible (and also Maven Integration if not installed).
12. Select both checkboxes and click Install.
13. Wait for installation to complete and restart Jenkins if required.

8.5 Step 4: Create Ansible Inventory File

Create an inventory file that specifies the target application server. All files are created under /home/ansible/ (not /home/user/):

Create inventory file (run as ansible user)

```
# Switch to ansible user first if needed
sudo su ansible

# Create the inventory file
nano /home/ansible/inventory.ini
```

Add the following content:

/home/ansible/inventory.ini

```
[appserver]
localhost ansible_connection=local
```

Save and exit: Ctrl+O, Enter to save, then Ctrl+X to exit.

Verify the file was created correctly:

Verify Inventory

```
cat /home/ansible/inventory.ini
```

Part	Meaning
[appserver]	Group name — Ansible will refer to this group in playbooks
localhost	The target host is the same machine (control node)
ansible_connection=local	Use local connection instead of SSH (no SSH needed)

8.6 Step 5: Write an Ansible Deployment Playbook

Create the deployment playbook at /home/ansible/deploy.yml (use full path, not relative path):

Create the playbook file

```
nano /home/ansible/deploy.yml
```

Enter the following content exactly:

/home/ansible/deploy.yml

```
- name: Deploy JAR artifact to appserver
  hosts: appserver
  become: yes
  vars:
    artifact_src: ~/jenkins-artifacts/myapp-1.0-SNAPSHOT.jar
    deploy_dir: /opt/myapp
  tasks:
    - name: Create deployment directory
      file:
        path: '{{ deploy_dir }}'
        state: directory
        mode: '0755'
```

```

- name: Copy artifact to deploy directory
  copy:
    src: '{{ artifact_src }}'
    dest: '{{ deploy_dir }}/myapp.jar'
    mode: '0644'

```

Save and exit: Ctrl+O, Enter, then Ctrl+X.

Section	Purpose
hosts: appserver	Targets the [appserver] group from the inventory file
become: yes	Run tasks with sudo/root privileges
artifact_src	Source path of the built JAR file (must exist before running)
deploy_dir	Destination directory on the server (/opt/myapp)
file module (Task 1)	Creates /opt/myapp directory if it does not exist
copy module (Task 2)	Copies the JAR from ~/jenkins-artifacts to /opt/myapp/myapp.jar

8.7 Step 6: Build the JAR and Prepare jenkins-artifacts

Before running the Ansible playbook, the JAR file must exist at ~/jenkins-artifacts/. Follow these steps:

Build the JAR manually (as ansible user)

```

# Navigate to the correct directory (where pom.xml is located)
cd ~/java-maven-app/myapp

# Verify pom.xml is present
ls pom.xml

# Build the JAR
mvn clean package

# Verify JAR was created
ls target/myapp-1.0-SNAPSHOT.jar

```

Copy JAR to jenkins-artifacts directory

```

# Create the directory if it does not exist
mkdir -p ~/jenkins-artifacts

# Copy the built JAR
cp ~/java-maven-app/myapp/target/myapp-1.0-SNAPSHOT.jar ~/jenkins-artifacts/

# Verify the file is in place
ls -la ~/jenkins-artifacts/myapp-1.0-SNAPSHOT.jar

```

Troubleshooting: Finding Your Jenkins Job Name

If Jenkins built the JAR, find where it was saved:

```
# List all Jenkins workspaces
ls /var/lib/jenkins/workspace/

# Find the JAR file directly (fastest method)
find /var/lib/jenkins/workspace/ -name '*.jar' 2>/dev/null
```

Then copy using the exact path returned by find.

8.8 Step 7: Run the Jenkins Build

14. From the Jenkins Dashboard, navigate to your Build-Maven-Project job.
15. Click Build Now.
16. Monitor the build progress in the Builds panel on the left.
17. Click on the build number (#1) and select Console Output to see the full build log.

Expected results in the console output:

- BUILD SUCCESS from Maven
- myapp-1.0-SNAPSHOT.jar is compiled and copied
- Archiving artifacts: myapp/target/*.jar
- Finished: SUCCESS

8.9 Step 8: Verify Deployment with Ansible

After the Jenkins build succeeds, run the Ansible deployment playbook. Always use full paths:

Run Ansible Deployment Playbook

```
ansible-playbook -i /home/ansible/inventory.ini /home/ansible/deploy.yml
```

Expected output:

Expected Playbook Output

```
PLAY [Deploy JAR artifact to appserver] *****
```

```
TASK [Gathering Facts] *****
ok: [localhost]
```

```
TASK [Create deployment directory] *****
changed: [localhost]
```

```
TASK [Copy artifact to deploy directory] *****
changed: [localhost]
```

```
PLAY RECAP *****
localhost : ok=3  changed=2  unreachable=0  failed=0  skipped=0
```

Success Indicator

If failed=0, your deployment was successful.

The JAR is now deployed at: /opt/myapp/myapp.jar

Verify the artifact was deployed:

Verify Deployment

```
ls -la /opt/myapp/
# Expected: myapp.jar present in the directory
```

8.10 Common Errors and Fixes

Error 1: deploy.yml could not be found

Fix: Use full path

```
# WRONG - relative path
ansible-playbook -i /home/ansible/inventory.ini deploy.yml

# CORRECT - full path
ansible-playbook -i /home/ansible/inventory.ini /home/ansible/deploy.yml
```

Error 2: Could not find ~/jenkins-artifacts/myapp-1.0-SNAPSHOT.jar

Fix: Build JAR and copy to jenkins-artifacts

```
cd ~/java-maven-app/myapp
mvn clean package
mkdir -p ~/jenkins-artifacts
cp target/myapp-1.0-SNAPSHOT.jar ~/jenkins-artifacts/
ls -la ~/jenkins-artifacts/
```

Error 3: Permission denied on /home/ansible

Fix: Grant Jenkins access to ansible home directory

```
sudo chmod o+rx /home/ansible
sudo chmod -R o+rx /home/ansible/java-maven-app

# OR add jenkins to ansible group (recommended):
sudo usermod -aG ansible jenkins
sudo systemctl restart jenkins
```

Error 4: No POM in this directory

Fix: Navigate to correct directory

```
# WRONG - no pom.xml here
cd ~/java-maven-app
```

```
mvn clean package    # FAILS

# CORRECT - pom.xml is inside myapp/
cd ~/java-maven-app/myapp
mvn clean package    # WORKS
```

8.11 Summary

Program 8 — Key Takeaways

- Always use full paths for inventory and playbook files with ansible-playbook.
- The JAR file must exist at ~/jenkins-artifacts/ BEFORE running the Ansible playbook.
- Maven pom.xml lives inside the 'myapp' subfolder — cd into it before running mvn.
- Jenkins runs as the 'jenkins' user — grant it access to /home/ansible/ via chmod or usermod.
- Jenkins Freestyle jobs use Execute shell build steps to run any shell command.
- The Archive the artifacts post-build action makes build outputs available in Jenkins.
- A complete CI/CD workflow automates: code build (Jenkins) + artifact deployment (Ansible).
- Console Output in Jenkins provides full visibility into build success or failure.

Program 8.0: Configuration Management with Ansible (Enhanced)

Topics Covered: Basics of Ansible — Inventory, Playbooks & Modules • Automating Server Configurations • Cleanup and Removal of Installed Packages

8.0.1 Removing Ansible and Cleaning Up

Step 1: Remove NGINX via Ansible Playbook

Create Removal Playbook

```
nano remove_nginx.yml
```

remove_nginx.yml

```
- name: Remove NGINX from localhost
  hosts: local
  become: yes
  tasks:
    - name: Stop NGINX service
      service:
        name: nginx
        state: stopped
    - name: Disable NGINX service
      service:
        name: nginx
        enabled: no
    - name: Remove NGINX package
      apt:
        name: nginx
        state: absent
        purge: yes
    - name: Remove residual config files
      file:
        path: /etc/nginx
        state: absent
```

Run Removal Playbook

```
ansible-playbook -i /home/ansible/hosts remove_nginx.yml
```

Step 2: Remove Ansible from the System

Option A: Remove Ansible Only (keep config files)

```
sudo apt remove ansible -y
```

Option B: Remove Ansible + Config Files

```
sudo apt purge ansible -y
```

Option C: Full Cleanup (Recommended)

```
sudo apt purge ansible -y
```

```
sudo apt autoremove -y
```

```
sudo apt autoclean
```

Step 3: Verify Ansible is Fully Removed

Verify Removal

```
ansible --version
```

```
# Expected: bash: ansible: command not found
```

Recommendation

Always use Option C (purge + autoremove + autoclean) to keep the lab environment clean and avoid leftover dependencies between experiments.

8.0.2 Summary

Program 9 — Key Takeaways

- Ansible uses an agentless SSH-based architecture for configuration management.
- The control node runs Ansible; managed nodes only need SSH and Python.
- Inventory files define the hosts Ansible manages (groups and individual hosts).
- Playbooks are YAML files containing named plays with tasks using modules.
- The ping module tests connectivity; apt and service modules manage packages and services.
- Run playbooks with: `ansible-playbook -i <inventory_file> <playbook.yml>`
- Use `state: absent` in apt/yum modules to cleanly remove packages via Ansible.

8.0.3 Viva-Voce Questions

18. What is Ansible and how does it differ from other configuration management tools like Puppet or Chef?
19. Explain the concept of idempotency in Ansible.
20. What is an Ansible Inventory file and what does it contain?
21. What is a Playbook? How is it different from an ad-hoc command?
22. What is the difference between apt remove, apt purge, and apt autoremove?
23. How does Ansible use SSH for agentless communication?
24. What does `state: absent` do in an Ansible apt module task?
25. How would you verify that a package has been successfully removed from a managed node?
26. Why must the group name in the inventory file match the `hosts:` field in the playbook?
27. What happens when you run `'sudo cd'` in a terminal? Why does it fail?

Appendix: Quick Reference

A. Common Jenkins Commands and URLs

Item	Value / Command
Jenkins Dashboard	<code>http://localhost:8080</code>
Jenkins Custom Port	<code>http://localhost:3030</code> (example)
Admin Password (Linux)	<code>/var/lib/jenkins/secrets/initialAdminPassword</code>
Admin Password (Windows)	<code>C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword</code>
Manage Jenkins	Dashboard > Manage Jenkins
Plugin Manager	Manage Jenkins > Plugins > Available
Global Tools	Manage Jenkins > Tools
New Job	Dashboard > New Item
Build History	Job > Build History (left sidebar)
Start Jenkins	<code>sudo systemctl start jenkins</code>
Stop Jenkins	<code>sudo systemctl stop jenkins</code>
Restart Jenkins	<code>sudo systemctl restart jenkins</code>
Remove Jenkins	<code>sudo apt purge jenkins -y && sudo rm -rf /var/lib/jenkins</code>

B. Common Ansible Commands

Command	Description
<code>ansible --version</code>	Show Ansible version info
<code>ansible all -m ping</code>	Ping all hosts in inventory
<code>ansible-inventory --list -y</code>	List inventory in YAML format
<code>ansible-playbook playbook.yml</code>	Run a playbook (default inventory)
<code>ansible-playbook -i hosts pb.yml</code>	Run a playbook with custom inventory
<code>ansible all -m command -a 'cmd'</code>	Run ad-hoc command on all hosts
<code>sudo apt install ansible -y</code>	Install Ansible on Ubuntu
<code>ssh-keygen</code>	Generate SSH key pair
<code>ssh-copy-id user@host</code>	Copy public key to remote host
<code>sudo apt remove ansible -y</code>	Remove Ansible (keep config)

<code>sudo apt purge ansible -y</code>	Remove Ansible + config files
<code>sudo apt purge ansible -y && sudo apt autoremove -y</code>	Full Ansible cleanup (recommended)

C. Ansible State Values for Packages and Services

Module	State Value	Action
apt / yum	present	Install the package
apt / yum	absent	Remove the package
apt / yum	latest	Install or upgrade to latest version
Service	started	Start the service
Service	stopped	Stop the service
Service	restarted	Restart the service
File	directory	Create a directory
File	absent	Delete a file or directory

About the Author

Dr. Lokesh M R

Professor —

Mangaluru, Karnataka

QUALIFICATIONS

Ph.D. Computer Science & Engineering

Sathyabama Institute of Science & Technology (2019)

M.Tech. Information Technology

National Institute of Engineering, Mysore (2007)

B.E. Electronics & Communication Engineering

PES College of Engineering, Mandya (2001)

ACADEMIC HIGHLIGHTS

22+ Years of Academic Experience

40+ Research Publications

2 Granted Patents

4 Active Ph.D. Scholars (VTU)

RESEARCH DOMAINS

- ▶ Cyber-Physical Systems & Fault Tolerance
- ▶ Artificial Immune Systems & Resiliency
- ▶ IoT Security & Smart Healthcare
- ▶ Machine Learning Applications
- ▶ Parallel & Cloud Computing
- ▶ Full Stack Development & DevOps

KEY COURSES TAUGHT

- ▶ Parallel Computing [BIS702] — 2025
- ▶ Full Stack Development [BIS61] — 2025
- ▶ Cloud Computing & Security [BIS658] — 2026
- ▶ DevOps Laboratory [BCSL657D] — 2026
- ▶ Data Visualization with Python — 2024

22+ Years of Academic Experience

2024–Now

Professor — IS&E, A J Institute of Engineering & Technology, Mangaluru

2023–24

Professor & Head — CS&E, Vivekananda College of Engg & Tech, Puttur

2019–22

Professor & Head — CS&E, Maharaja Institute of Technology, Mysore

2018–19

Assoc. Prof & Head — CS&E, Mysore Royal Institute of Technology, Mandya

Pre-2018

Asst. Prof — GSSSIETW Mysore | NHCE Bengaluru | DSI Bangalore | TCS

Contact

✉ lokeshmrmysore@gmail.com

☎ +91 99640 50926

Scopus · ORCID · Google Scholar · Vidwan

Professional Memberships

- Computer Society of India — Life Member (2019)
 - ISTE — Life Member (2019)
- VTU Approved Ph.D. Research Supervisor

© 2026 Dr. Lokesh M R · Mangaluru · All Rights Reserved

© 2026 Dr. Lokesh M R | Mangaluru | All Rights Reserved